

CareerPilot

PLAN · PREPARE · PERFORM · SUCCEED

Mid-Evaluation Progress Review

Karan Rana · B.Tech Computer Science & Engineering · 2nd Year

Madhav Institute of Technology and Science, Gwalior

github.com/karan-rxna/Career-Pilot

CareerPilot

CONTENTS

What this review covers

Ten areas, from the problem to the plan for the next milestone.

01 Introduction

02 Problem Statement

03 Objectives

04 Motive

05 Technology Stack

06 System Architecture

07 Database Design

08 Authentication & Security

09 Current Status

10 Product Walkthrough

11 Roadmap & Future Scope

12 Conclusion

01 · INTRODUCTION

One workspace for the whole placement journey

CareerPilot connects to the platforms a student already uses, reads the progress already there, and turns it into analytics, a plan and a portfolio.

Existing tools stop at listing opportunities. CareerPilot starts where they stop — with the student's own activity.

Portals only list

Placement portals publish openings, but offer no path from 'interested' to 'ready'.

Trackers make you type

DSA trackers ask the student to mark problems solved by hand, so they go stale within a week.

Evidence is scattered

Solved counts, contest ratings and commits live on separate platforms with no single view.

The answer: **one dashboard that reads what you have already done and plans what to do next.**

02 · PROBLEM STATEMENT

Preparation is fragmented, and progress is invisible

Opportunities without preparation

Placement portals post jobs and internships, but nothing helps a student become the candidate who gets shortlisted.

Manual, self-reported DSA tracking

Every DSA tracker depends on the student marking problems solved — and rots the moment they stop.

No unified progress view

Platform statistics live in silos, so weak topics stay invisible until an interview exposes them.

Scattered placement material

Contests, hackathons, roadmaps, notes and interview preparation sit in unrelated tools and browser tabs.

The gap: students are already generating the data — it is simply never assembled into a picture they can act on.

03 · OBJECTIVES

Six concrete objectives for version 1

01

Unify analytics

One dashboard for DSA progress, GitHub activity and contest performance.

02

Auto-sync progress

Connect a profile and statistics update themselves — no manual entry anywhere.

03

Plan the work

Goals, roadmaps and a daily plan derived from real activity.

04

Surface opportunities

Contests and hackathons, searchable, filterable and bookmarkable.

05

Centralise knowledge

A notes vault and a categorised interview-preparation hub.

06

Be AI-ready

An architecture where recommendations need no rework to switch on.

04 · MOTIVE

Why I am building this

Built from the inside

As a 2nd-year CSE student preparing for placements, every problem in this deck is one I have hit myself.

Practice without proof is invisible

A recruiter sees a résumé, not the 300 problems behind it. Progress needs to be presentable.

Consistency needs feedback

Streaks, heatmaps and weekly analytics turn daily practice into something sustainable.

> **“The problem was never a lack of effort — it was never having one place to see it.”**
>
> That sentence is the whole product requirement.

05 · TECHNOLOGY STACK

Chosen for the job each layer actually does

Frontend

- React 19
- Vite 8
- TypeScript
- Tailwind CSS v4
- React Router 7
- Framer Motion
- Recharts
- lucide-react

Type-safe SPA with a component model that keeps 11 feature screens maintainable.

Backend

- Java 21
- Spring Boot 4.1
- Spring Security
- JWT (HS256)
- Spring Data JPA
- Bean Validation
- Flyway
- Maven

Batteries-included REST API: dependency injection, security and transactions built in.

Data

- PostgreSQL 17 (Neon)
- Hibernate · validate
- 23 tables
- Flyway V1 + V2
- 127 seed rows
- HikariCP pooling
- TIMESTAMPTZ (UTC)
- JSONB snapshots

Managed Postgres with branching and scale-to-zero; Flyway owns every schema change.

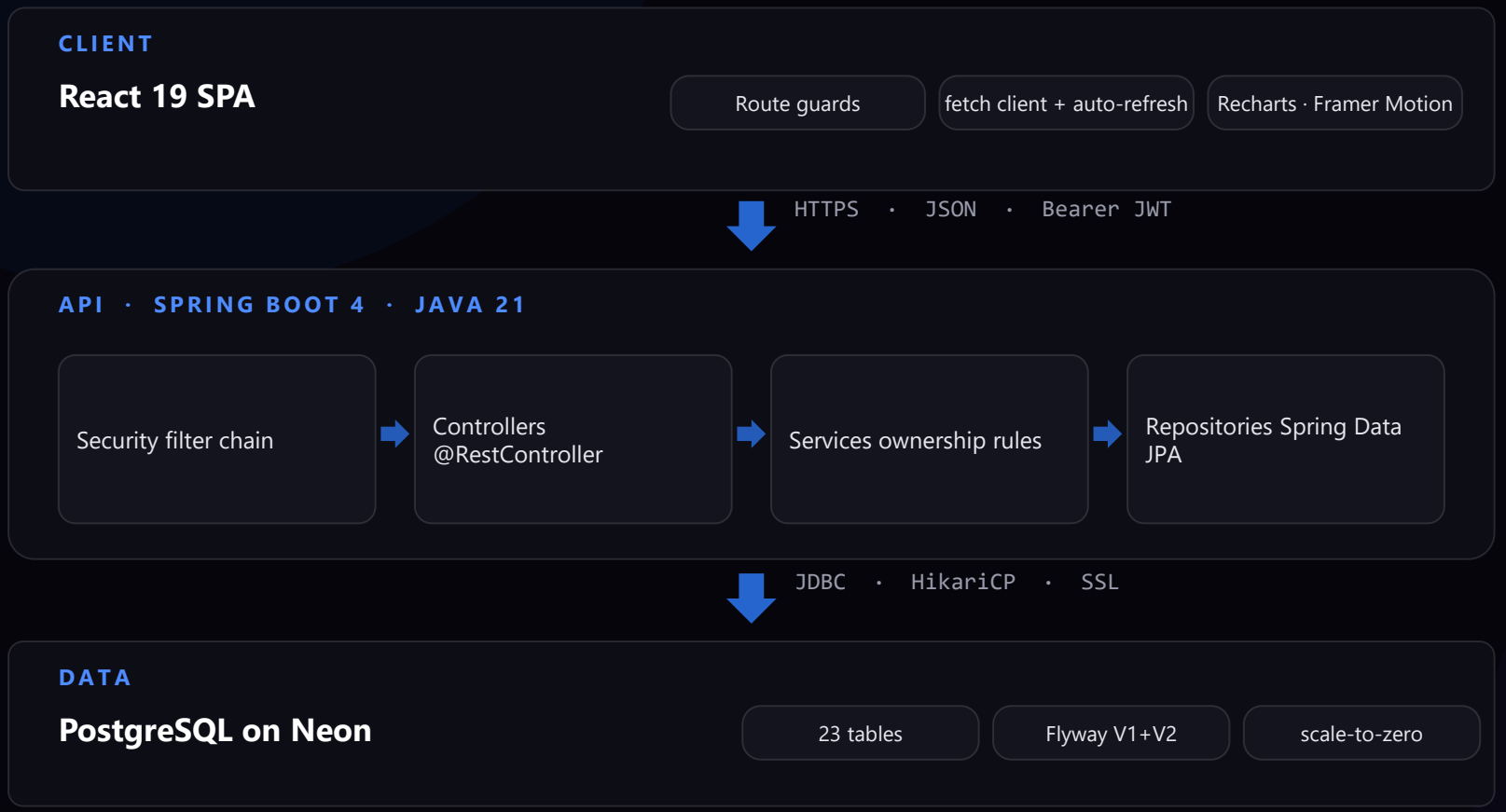
Delivery

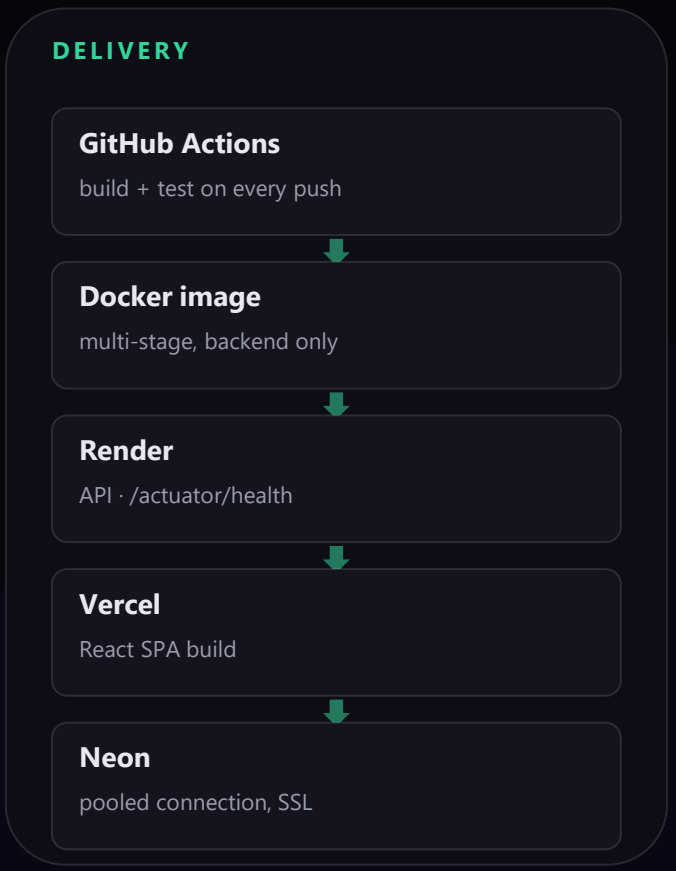
- Docker (multi-stage)
- GitHub Actions CI
- Render (API)
- Vercel (SPA)
- Actuator health
- Neon pooled URL

Both halves deploy from main; a free-tier stack that still mirrors production.

06 · SYSTEM ARCHITECTURE

Three tiers, one contract





07 · DATABASE DESIGN

23 tables, all under migration control

IN USE TODAY

users

id · email (unique) · password_hash · name · handle · college · batch · headline · readiness_score · role · timestamps

user_skills

user_id → users · skill · proficiency — the profile's skill list

refresh_tokens

token_hash · user_id · expires_at · revoked_at — rotated on every use

password_reset_tokens

token_hash · user_id · expires_at · used_at — single-use, 60 min

DESIGNED & SEEDED — 19 MORE

Coding & portfolio

platform_accounts · coding_snapshots · topic_stats · repos · activity_events

Productivity

goals · notes · bookmarks · daily_plans · daily_plan_items

Learning

roadmaps · roadmap_stages · roadmap_resources · roadmap_progress

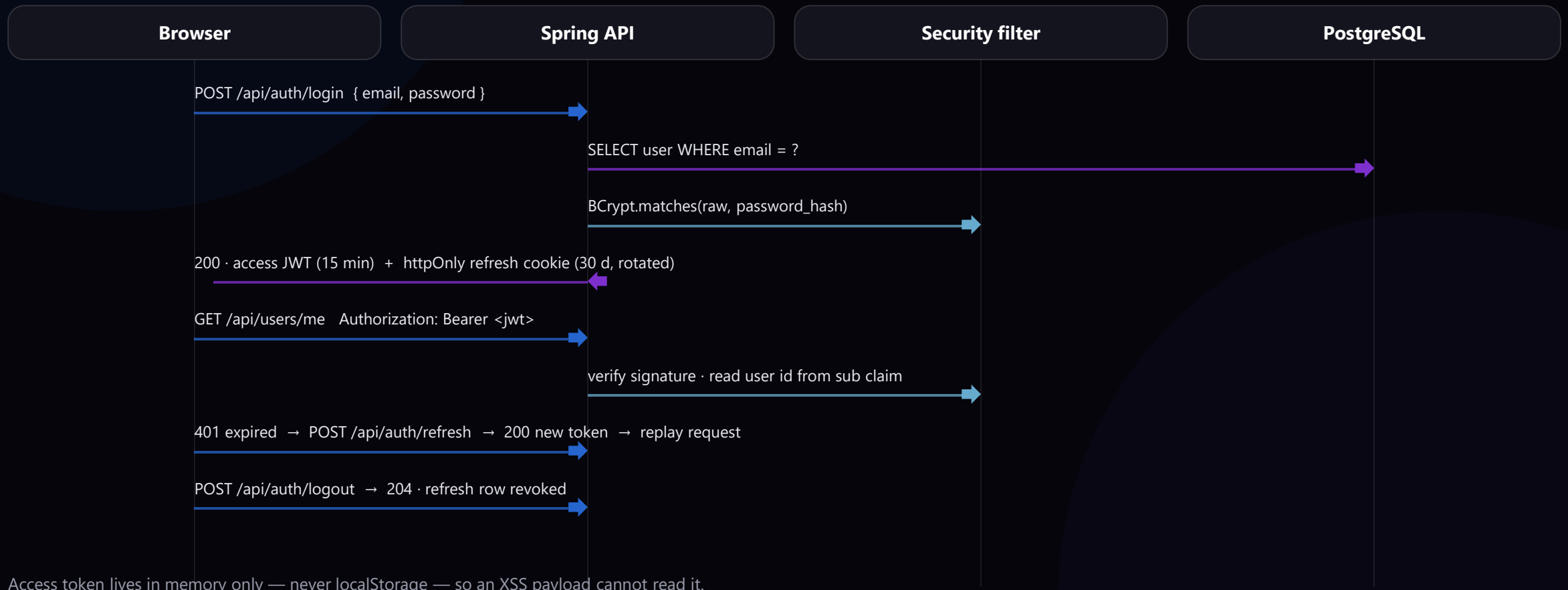
Interview & discovery

interview_categories · interview_topics · interview_progress · contests · hackathons

23 tables | 2 Flyway migrations | 127 seed rows | Flyway owns DDL | ddl-auto = validate | TIMESTAMPTZ stored in UTC

08 · AUTHENTICATION & SECURITY

Login, then a session that survives a reload



Access token lives in memory only — never localStorage — so an XSS payload cannot read it.

09 · CURRENT STATUS

What is verified, what is seeded, what is next

Stated precisely — the deeper feature screens still render mock data by design.

LIVE & VERIFIED

- Register · login · refresh · logout
- Forgot & reset password (single-use)
- GET · PATCH /api/users/me
- GET /api/users/me/stats
- 23-table schema applied on Neon
- BCrypt ($2a$10$) password hashing
- JWT + rotated httpOnly refresh cookie
- Route guards · 401 for anonymous
- GitHub Actions CI green

DESIGNED & SEEDED

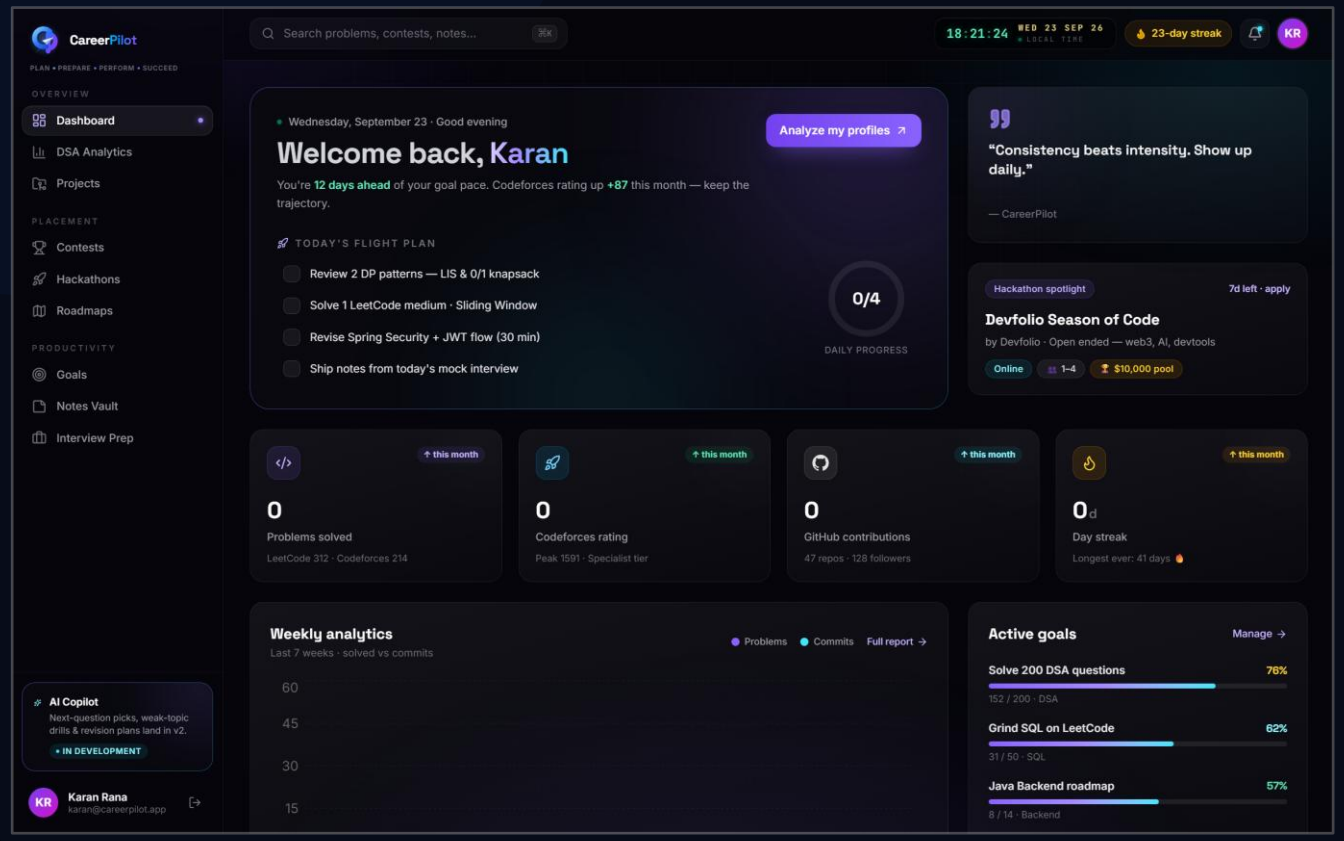
- goals · notes · bookmarks
- daily_plans · daily_plan_items
- roadmaps (+ stages, resources, progress)
- interview categories, topics, progress
- contests · hackathons
- 127 seed rows already loaded
- Endpoints defined in the API contract
- Frontend screens already built
- Mock data layer still in use

NEXT MILESTONES

- Goals vertical slice (real DB reads/writes)
- Feature fan-out across all 11 sections
- Platform sync: GitHub · Codeforces · LeetCode
- Rule-based recommendations + daily plan
- AI layer: portfolio review, README, bullets
- Retire mock.ts entirely
- Production deploy: Vercel + Render
- Rate limits, monitoring, backups
- Root README + LICENSE

10 · PRODUCT WALKTHROUGH

The dashboard: everything in one screen



Real screenshot from the running build — the sidebar, top bar and avatar all resolve from the live API.

Welcome & pace

Goal pace, streak and a nudge, computed from the week's activity.

Daily flight plan

A short, achievable checklist for today — not a backlog.

Stat tiles

Problems solved, contest rating, GitHub contributions, day streak.

GitHub heatmap

Contribution graph with longest streak and top language.

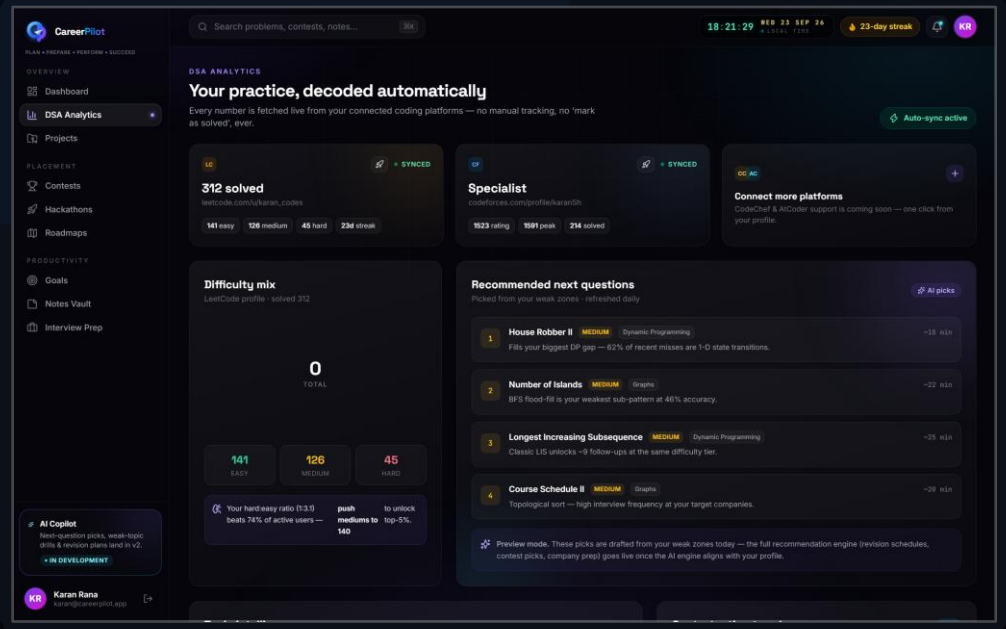
Contests & roadmaps

What's coming up, and where you are in each learning path.

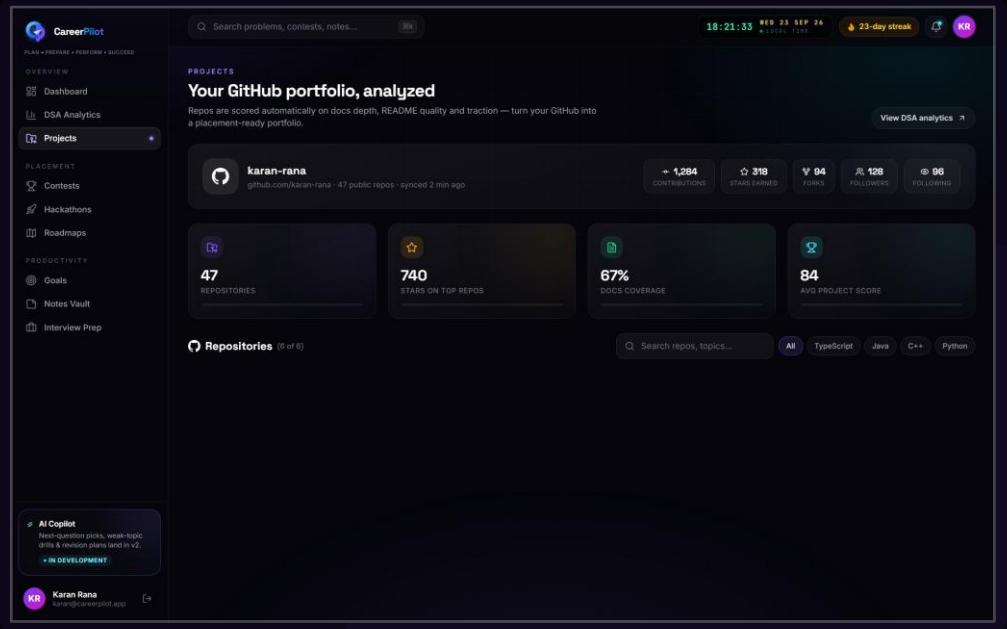
10 · PRODUCT WALKTHROUGH

DSA analytics and the GitHub portfolio analyzer

These two pages replaced a single combined 'coding analytics' screen — the data they show has different refresh rates and different owners.



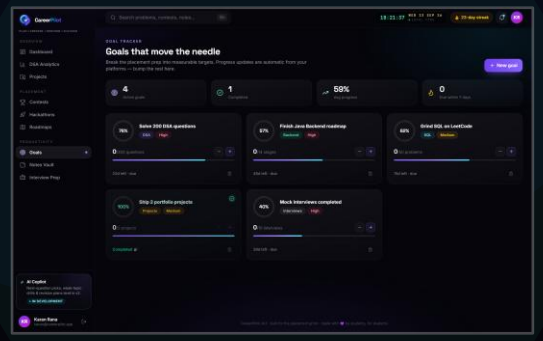
DSA Analytics — platform cards per profile, difficulty split, topic intelligence with weak-topic flags, Codeforces rating trend, practice heatmap, and a reserved card for AI recommendations.



Projects — GitHub statistics, a repo grid with live-demo and source links, language breakdown, and portfolio-quality scoring across documentation and structure.

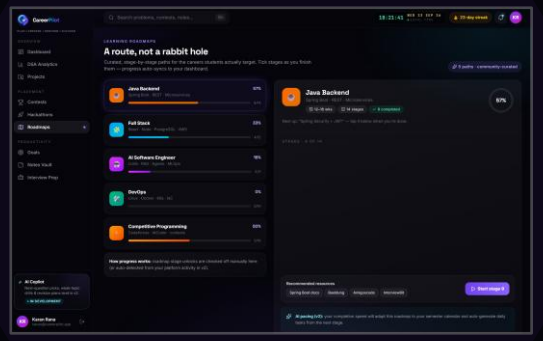
10 · PRODUCT WALKTHROUGH

Planning and discovery surfaces



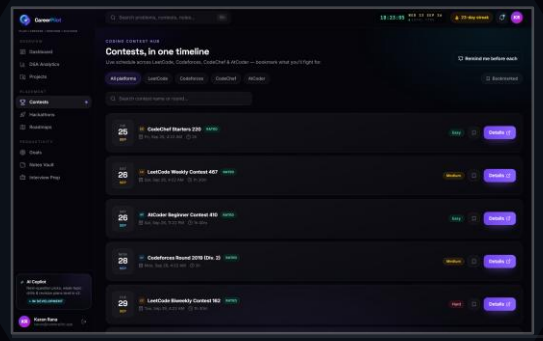
Goals

Progress, deadlines, priority and status — with completion percentages that update from linked activity.



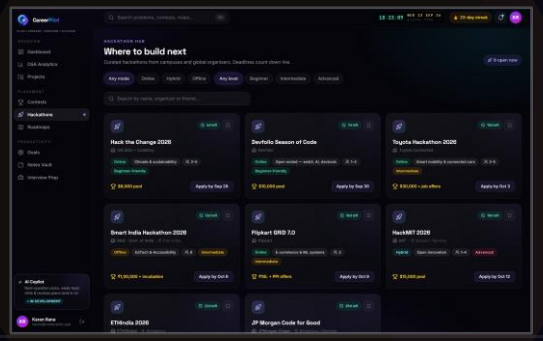
Roadmaps

Curated Java Backend, Full Stack, AI, DevOps and Competitive Programming paths with staged progress.



Contests

A single timeline across LeetCode, Codeforces, CodeChef and AtCoder, with start times and bookmarks.



Hackathons

Search and filter by mode, team size, theme and prize pool, with bookmarks and admin management.

11 · ROADMAP & FUTURE SCOPE

Where the build stands, stage by stage

Stages 0–3 complete · roughly 40% of the planned build

8–10 weeks part-time remaining

STAGE 0	STAGE 1	STAGE 2	STAGE 3	STAGE 4
Contracts & docs	Database & migrations	Backend skeleton	Auth & identity	Goals vertical slice
Verified	Verified	Verified	Verified	In progress

STAGE 5	STAGE 6	STAGE 7	STAGE 8	STAGE 9
Feature fan-out	Platform sync engine	Recommendations	AI layer	Hardening & ship
Planned	Planned	Planned	Planned	Planned

Milestone 1 (Stages 0–3) is complete and verified. Milestone 2 begins with the Goals vertical slice, which proves the full browser → API → database path on real data.

11 · FUTURE SCOPE

What version 2 unlocks

AI recommendation engine

Next question to solve, weak-topic drills, a weekly practice plan and a revision schedule.

Live platform sync

Codeforces, GitHub and LeetCode pulled on a schedule into time-series snapshots, with a 'synced X ago' badge.

AI portfolio review

Resume bullets, README suggestions and project improvements generated from real repository data.

Career readiness score

A single score built from solved problems, contest rating, portfolio quality and interview coverage.

Harden & scale

Rate limits, lazy-loaded routes to cut the bundle, monitoring, PITR backups and a separate production environment.

The AI layer is deliberately last, and deliberately optional. Stage 7 is a rule-based engine that produces the same recommendations deterministically — so recommendations work, and keep working, with no model in the loop. AI then explains and personalises them; it never invents the numbers.

12 · CONCLUSION

What this milestone establishes

The foundation is real

A 23-table schema on managed Postgres, a secured REST API and a complete authentication lifecycle — all running, not sketched.

The contract is frozen

Every endpoint, request body and error code for all eleven feature areas is written down, so features are built against an agreed interface.

The UI is ahead by design

Eleven screens are already built against that contract, which is why swapping mock data for live data is a data-layer change, not a rewrite.

CareerPilot is not another opportunity board or manual DSA tracker — it is the workspace that reads what a student has already done and turns it into a plan.

Next: finish the feature slices on this foundation, then switch on the recommendation engine.



Thank you

Questions and discussion

Karan Rana · B.Tech Computer Science & Engineering · 2nd Year

Madhav Institute of Technology and Science, Gwalior

github.com/karan-rxna/Career-Pilot